CYBER-SENTINEL: A Portable ESP32-Based Rogue Access Point Detector and LightWeight HoneyPot

1. Introduction

Wireless connectivity has become the primary means of network access in homes, campuses, and enterprises. This convenience, however, comes with an inherent security weakness, Wi-Fi management frames such as beacons and probe responses are transmitted without strong authentication in most consumer deployments, allowing an attacker to impersonate a legitimate access point with commodity hardware.

A Rogue Access Point is any unauthorized access point connected to a network. The most dangerous attack is the Evil Twin attack, in which an attacker configures an access point with same SSID as the legitimate network and frequently sends a strong signal to connect with the client devices. Once a victim connects the attacker can perform traffic interception, credential harvesting and MITMs due to where most of the consumer devices display only the network name but not the BSSID which had been assigned. Users also have little practical way to distinguish a genuine AP from an impersonation at the moment of connection.

The commercial Intrusion Detection Systems address this problem but typically require dedicated sensor hardware, licensed software and centralized infrastructure that had be placing them out of reach of individual researcher's small offices and students who wish to study and monitor the wireless threats in the network practically. This requires the most affordable and easily deployable device capable of performing baseline rogue AP detection.

Why ESP32 Had been used in this Project?

The EXP32 micro controller was selected as the platform for this project because it integrates a dual mode WIFI radio sufficient processing for scanning and comparison logic and rich set of the GPIO peripherals for the user feed and very low unit cost relative dedicated wireless security appliance. These Characteristics make it well suitable for battery powered handheld monitoring tool.

2. Objective

- To design and implement the portable battery powered wifi scanning device using ESP32.
- To detect the potential rogue access point and Evil Twin conditions by comparing the BSSID, RSSI and the channel information against a trusted AP baseline.
- To provide the real time alerting through an OLED display, LED indicators and a active buzzer.
- To support the local logging of scan results for offline analysis.
- To outline a conceptual extension of the platform into a lightweight honeypot system in conjunction with companion single board computer.

3. Hardware and Software Implementation

Hardware Components

- ESP32 development board – dual core microcontroller with the integrated 802.11 b/g/n Wi-Fi used CPU and scanning unit.
- 0.96-inch OLED display – used to present scan results trusted or suspicious AP status and system messages.
- Active Buzzer – provides an audible alert when suspicious AP is detected.
- Red LED – visual threat indicator illuminated during an active alert.
- Green LED – visualize the safe status indicator illuminated when there is no threats.
- Bread Board and Jumper wires – used for prototyping the circuit.
- MicroSD card module – enables persistent logging of scan results for later offline analysis.

Software Components

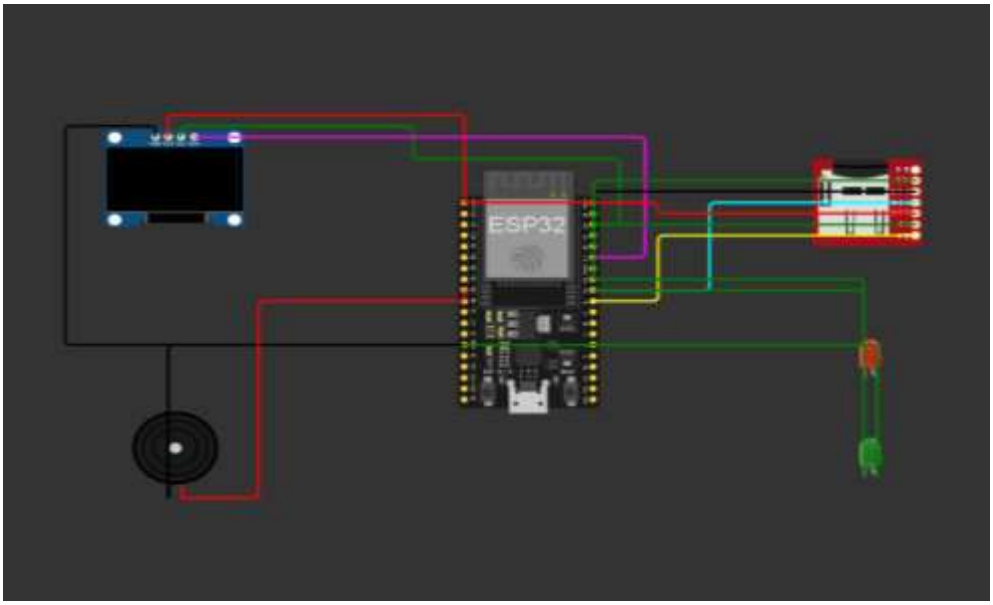
- Arduino IDE – development environment used to write, compile and upload firmware to the ESP32.
- ESP32 Arduino core framework – provides the underlying hardware abstraction and Wi-Fi APIs.
- C/C++ - implementation language for the firmware.

- WiFi.h – used to perform network scanning and retrieve SSID, BSSID, RSSI and channel data.
- Wire.h – I2C communication library used to drive the OLED display.
- Adafruit GFX and Adafruit SSD1306 libraries – provide graphics primitives ND OLED display driving routines.
- SD library – enables writing scan logs to MicroSD card wen the logging module is flitted.

Pin Configuration

Peripheral	Signal	ESP32 Pin
OLED(SSD1306)	VCC	3.3 V
OLED(SSD1306)	GND	GND
OLED(SSD1306)	SDA	GPIO21
OLED(SSD1306)	SCL	GPIO22
Buzzer	Positive	GPIO25
Buzzer	Negative	GND
Red LED	Anode	GPIO18
Red LED	Cathode	GND
Green LED	Anode	GPIO 19
Green LED	Cathode	GND
MicroSD	CS	GPIO5
MicroSD	SCK	GPIO18
MicroSD	MOSI	GPIO23
MicroSD	MISO	GPIO19
MicroSD	VCC	3.3V
MicroSD	GND	GND

Circuit Diagram



4. Source Code

```
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

#define BUZZER 25
#define RED_LED 18
#define GREEN_LED 19

void setup() {

  Serial.begin(115200);

  pinMode(BUZZER, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);

  digitalWrite(BUZZER, LOW);

  Wire.begin(21,22);
```

```
26.
27.  if(!display.begin(SSD1306_SWITCHCAPVCC,0x3C)){
28.    Serial.println("OLED Failed");
29.    while(true);
30.  }
31.
32.  display.clearDisplay();
33.  display.setTextSize(1);
34.  display.setTextColor(WHITE);
35.
36.  WiFi.mode(WIFI_STA);
37.  WiFi.disconnect();
38.
39.  delay(1000);
40.
41.  display.println("Cyber-Sentinel");
42.  display.println("Initializing...");
43.  display.display();
44.
45.  delay(2000);
46.}
47.
48.void loop() {
49.
50.  display.clearDisplay();
51.  display.setCursor(0,0);
52.
53.  display.println("Scanning...");
54.  display.display();
55.
56.  int n = WiFi.scanNetworks();
57.
58.  int threats = 0;
59.
60.  Serial.println("---------------------------------");
61.
62.  for(int i=0;i<n;i++){
63.
64.      String ssid1 = WiFi.SSID(i);
65.      String mac1  = WiFi.BSSIDstr(i);
66.
67.      Serial.print(ssid1);
68.      Serial.print("  ");
69.      Serial.print(mac1);
70.      Serial.print("  ");
71.      Serial.println(WiFi.RSSI(i));
72.
73.      for(int j=i+1;j<n;j++){
```

```
74.
75.           String ssid2 = WiFi.SSID(j);
76.           String mac2  = WiFi.BSSIDstr(j);
77.
78.           if(ssid1==ssid2 && mac1!=mac2){
79.
80.               threats++;
81.
82.               Serial.println("***** POSSIBLE ROGUE AP *****");
83.               Serial.println(ssid1);
84.
85.               digitalWrite(RED_LED,HIGH);
86.               digitalWrite(GREEN_LED,LOW);
87.
88.               tone(BUZZER,2000);
89.               delay(300);
90.               noTone(BUZZER);
91.
92.           }
93.
94.       }
95.
96.   }
97.
98.   if(threats==0){
99.
100.            digitalWrite(RED_LED,LOW);
101.            digitalWrite(GREEN_LED,HIGH);
102.
103.        }
104.
105.        display.clearDisplay();
106.
107.        display.setCursor(0,0);
108.        display.println("Cyber Sentinel");
109.        display.println();
110.
111.        display.print("WiFi : ");
112.        display.println(n);
113.
114.        display.print("Threats : ");
115.        display.println(threats);
116.
117.        if(threats>0)
118.            display.println("Status: ALERT");
119.        else
120.            display.println("Status: SAFE");
121.
```

```
122.         display.display();
123.
124.         delay(10000);
125.
126.       }
```

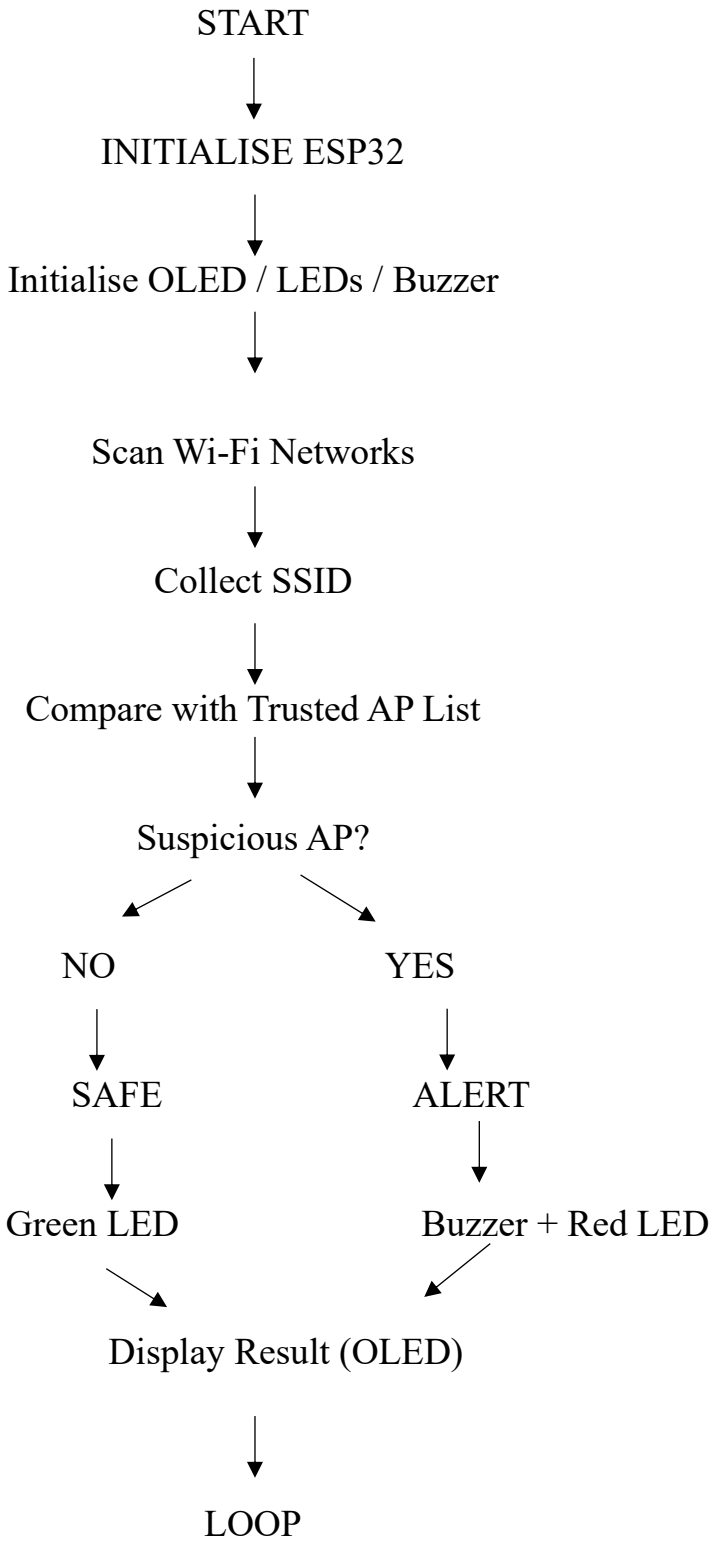
5. Wi-Fi Scanning and Processing Logic

On each scan cycle, the firmware invokes the ESP32 WiFi scan routine in station mode, which returns the number of visible access points along with their SSID, BSSID, RSSI channel and encryption type. The firm firmware iterates over the result set and, for each access point, checks whether its SSID matches an entry in the trusted-AP list stored in flash memory. If a match is found, the BSSID of the observed AP is compared against the BSSID recorded for that trusted entry. A mismatch increments a suspicion score for that SSID, which is combined with the channel and RSSI comparisons described in Section 3 before a final classification is passed to the output-handling routine that drives the OLED, LEDs, and buzzer.

6. ROGUE AP Detection Algorithm

- Initialise the ESP32, OLED display, LEDs, buzzer, and push button.
- Load the trusted-AP list (SSID and corresponding BSSID) from flash memory.
- Perform a Wi-Fi scan and collect SSID, BSSID, RSSI, and channel for each visible access point.
- For each scanned AP, search the trusted-AP list for a matching SSID.
- If a matching SSID is found, compare the observed BSSID, channel, and security type against the trusted entry.
- If the BSSID differs, classify the AP as suspicious; otherwise classify it as safe.
- Update the OLED display with the scan summary and classification.
- If suspicious, activate the red LED and buzzer; if all APs are safe, keep the green LED active.
- If MicroSD logging is enabled, append the result to the log file.
- Wait for the configured scan interval and repeat from Step3.

7. Flowchart


START
INITIALISE ESP32
Initialise OLED / LEDs / Buzzer
Scan Wi-Fi Networks
Collect SSID
Compare with Trusted AP List
Suspicious AP?
NO
YES
SAFE
ALERT
Green LED
Buzzer + Red LED
Display Result (OLED)
LOOP

7. Limitations

- The ESP32 has limited CPU, RAM, and storage compared with a Raspberry Pi or computer, restricting advanced packet analysis and complex security algorithms.
- The ESP32 cannot practically run advanced Linux-based honeypots such as Cowrie, Dionaea, or full SSH/FTP deception environments
- The system may incorrectly identify legitimate APs as suspicious or fail to detect sophisticated rogue Aps.
- Modern devices may use randomized MAC addresses, making reliable device identification more difficult.
- An Evil Twin can imitate legitimate network characteristics, so detection based only on SSID, BSSID, RSSI, or channel information cannot guarantee identification
- Continuous Wi-Fi scanning consumes power, reducing battery life in a portable implementation
- Detection accuracy can be improved significantly by maintaining an accurate list of trusted SSIDs and BSSIDs. Incorrect configuration can increase false alerts.

8. Conclusion

This project has presented the design of Cyber-Sentinel, a portable, ESP32-based device for detecting potential rogue access points and Evil Twin conditions in Wi-Fi environments. By comparing observed SSID, BSSID, channel, and signal characteristics against a trusted-AP baseline, and communicating findings through an OLED display, LED indicators, and a buzzer, the system provides an accessible entry point into practical wireless security monitoring at a fraction of the cost of commercial WIDS solutions. The project's cybersecurity value lies not in replacing enterprise-grade monitoring, but in demonstrating, at low cost, the principles underlying rogue-AP detection and in providing a usable field tool for students, researchers, and small-network administrators.